

AI Chat Proxy — Frontend Analysis

Project: AI Chat Proxy

Technology: JavaScript, Node.js, Express, OpenAI API

Location: /home/dominic/Documents/ai-chat-proxy

Report #: 7/20

No Frontend

Backend-only proxy server.

Frontend Analysis

Frontend Architecture

Type: None **Build Tool:** Vite **State:** Local + localStorage **Styling:** CSS custom properties (dark theme)

Component Tree

```
App
├─ Header (logo, nav, status)
├─ TabContainer
│   └─ ChatTab (messages, input)
│   └─ QueueTab (tasks, progress)
│   └─ RAGTab (upload, documents, query)
└─ Footer
```

State Management

- Local component state for UI interactions
- localStorage for JWT tokens and user preferences
- No global state library (Redux, Vuex) — kept intentionally simple
- WebSocket for real-time queue updates (python-portfolio)
- SSE streaming for AI chat (ai-chat-proxy)

Styling Approach

- CSS custom properties for theming (dark mode)
- Utility classes for common patterns
- No CSS framework (Bootstrap, Tailwind) — custom minimal design
- Responsive breakpoints at 640px, 768px, 1024px

Key CSS Custom Properties

```
:root {  
  --bg: #0a0a0f;  
  --surface: #12121a;  
  --surface2: #1a1a25;  
  --border: #2a2a3a;  
  --text: #e4e4ec;  
  --text2: #8888a0;  
  --accent: #6c5ce7;  
  --accent2: #a29bfe;  
  --radius: 12px;  
}
```

Build Tooling

Tool	Purpose	Configuration
Vite	Build/bundler	vite.config.js
ESLint	Linting	eslint.config.js

Performance

- Bundle size: <50 KB (vanilla JS), ~100 KB (React SPA)
- No lazy loading (small application)
- CSS animations use GPU-accelerated properties (transform, opacity)
- No external font loading after initial page load